

DEPARTMENT OF COMPUTER SCIENCE • SINGLE SUBJECT SERIES

CS-101

PROGRAMMING FUNDAMENTALS MASTER TEXTBOOK

Comprehensive 23-Page University Lecture Notes, Theory Foundations &
Solved Exam Question Bank

OFFICIAL 23-PAGE SINGLE SUBJECT EDITION • VERIFIED ACADEMIC MASTERPIECE

CURRICULAR SCOPE & CORE MODULES INCLUDED:

- Primitive Data Types, Overflow, & Type Casting
- Control Structures: if/else, switch, loops (for, while, do-while)
- Functions, Call Stack, Scope, Recursion, Pass-by-Value vs Reference
- Arrays (1D & 2D), Row-Major Matrix Operations & C-Strings
- Pointers, Pointer Arithmetic, Dynamic Memory Allocation (new/delete)

AUTHOR & PUBLICATION METADATA

Author / Curator: Asad Usman (CS Expert)

Curriculum Year: 2026 - 2027 Academic Session

Department: Department of Computer Science

Verification: Peer-Reviewed CS Hub

Academic Series: Undergraduate Subject Handbook • Edition 2026 Status: Full Academic Release

SYLLABUS**CS-101: Programming Fundamentals**

Foundations of imperative problem solving, structured programming, memory management, pointers, and algorithmic logic in C++.

COURSE OVERVIEW

A rigorous exploration of computational thinking, imperative programming paradigms, machine-level execution, pointer arithmetic, dynamic memory heaps, and algorithm design in C++.

1.0 Accredited Learning Outcomes**Outcome 1**

Deconstruct complex real-world algorithmic problems into deterministic structured logic.

Outcome 2

Trace execution state across stack frames, register files, and dynamic heap allocations.

Outcome 3

Eliminate dangling pointers, buffer overflows, and memory leaks using deterministic lifecycle management.

Outcome 4

Design modular, performant applications utilizing pointers, structs, and binary file streams.

NAVIGATION

Comprehensive Table of Contents

Detailed Index across all 23 Pages of CS-101

Subject Module / Chapter Title	Page
Course Overview & Program Learning Outcomes	Page 2
Detailed Table of Contents	Page 3
Chapter 1: Foundational Theory & Architectural Primitives	Pages 4 - 6
Chapter 2: Structural Mechanics & Idiomatic Implementation	Pages 7 - 9
Chapter 3: Advanced Optimization & Complexity Analysis	Pages 10 - 12
Chapter 4: Complete University Slide Deck Notes (Part 1)	Pages 13 - 14
Chapter 5: Complete University Slide Deck Notes (Part 2)	Pages 15 - 16
Chapter 6: Midterm & Final Solved Examination Bank	Pages 17 - 18
Chapter 7: Semester Capstone Lab Specification & Rubric	Pages 19 - 20
Chapter 8: FAANG Technical Interview Preparation Guide	Pages 21 - 22
Chapter 9: Subject Academic Lexicon & Authoritative Bibliography	Page 23

CHAPTER 1**Machine Architecture, Compilation Pipelines & Memory Representation**

From High-Level Source to CPU Microcode Execution

Understanding computing starts at the silicon boundary. Source code is not executed directly; it undergoes four transformations: preprocessor macro expansions, syntactic translation into assembly, machine code object translation, and link-time address resolution. At runtime, the operating system assigns virtual address spaces partitioned into Text, Data, BSS, Heap, and Stack segments.

Theoretical Foundation 1.1

The Von Neumann Architecture bottleneck: Shared memory bus between CPU program instructions and data operands.

Theoretical Foundation 1.2

Virtual Memory Segments: The Text segment contains immutable CPU opcodes; Data/BSS host global and static variables.

Theoretical Foundation 1.3

Endianness: Little-endian systems store the least significant byte at the lowest memory address.

Theoretical Foundation 1.4

Primitive representations: IEEE 754 floating point standard (sign bit, biased exponent, mantissa precision).

SOURCE LISTING: Memory Address Inspection & Byte Serialization (C++)

```
#include <iostream>
#include <iomanip>

void inspectBytes(const void* ptr, size_t size) {
    const unsigned char* bytePtr = static_cast<const unsigned char*>(ptr);
    std::cout << "Memory Dump at " << ptr << ": ";
    for (size_t i = 0; i < size; ++i) {
        std::cout << "0x" << std::hex << std::setw(2) << std::setfill('0')
            << static_cast<int>(bytePtr[i]) << " ";
    }
    std::cout << std::dec << "\n";
}

int main() {
    int val = 0x12345678;
    std::cout << "Integer value: " << val << "\n";
    inspectBytes(&val, sizeof(val)); // Demonstrates Little-Endian byte ordering
    return 0;
}
```

TECHNICAL ANALYSIS

This snippet casts any generic memory address to an unsigned byte pointer, exposing how 32-bit integers are stored across four consecutive byte addresses in x86/ARM hardware architectures.

Data Type	Size (x86_64)	Value Range	Typical Alignment
char	1 Byte	-128 to +127	1-byte boundary
short	2 Bytes	-32,768 to +32,767	2-byte boundary
int	4 Bytes	-2.14B to +2.14B	4-byte boundary
long long	8 Bytes	-9.22E18 to +9.22E18	8-byte boundary
float	4 Bytes	~7 decimal digits precision	4-byte boundary
double	8 Bytes	~15-17 decimal digits	8-byte boundary
pointer (T*)	8 Bytes	0x0000000000000000 to 0xFFFFFFFFFFFFFFFF	8-byte boundary

SAMPLE EXAM QUESTION: Midterm Standard

What is the precise difference between a compiler error and a linker error? Provide concrete examples of code triggering each.

SOLUTION & KEY TAKEAWAY

A compiler error occurs during lexical analysis, parsing, or semantic checking when C++ syntax rules or type constraints are violated (e.g., missing semicolons, undeclared variables, or mismatched parameter types). A linker error occurs after individual translation units (.cpp -> .o) are compiled, when the linker attempts to bind unresolved symbols into a final executable (e.g., calling a function prototyped in a header whose definition is missing, or having multiple global definitions violating the One Definition Rule).

CHAPTER 2

Control Structures, Loop Invariants & Algorithmic Branching

Branch Prediction, Short-Circuiting, and Recursion Semantics

Conditional branching governs execution paths. Modern superscalar CPUs utilize hardware branch predictors with history tables to speculatively execute instructions. High-performance code must account for pipeline stalls induced by branch mispredictions, short-circuit boolean semantics, and formal loop invariants.

Theoretical Foundation 2.1

Branch Prediction: Modern CPUs execute ahead based on 2-bit saturating counters; mispredictions incur 15-20 cycle pipeline flushes.

Theoretical Foundation 2.2

Short-Circuit Evaluation: In $(A \ \&\& \ B)$, if A evaluates to false, B is guaranteed never to execute, crucial for guard clauses like $(ptr \neq \text{nullptr} \ \&\& \ ptr->\text{val} > 0)$.

Theoretical Foundation 2.3

Loop Invariant: A formal mathematical assertion that holds true before loop entry, at every iteration boundary, and immediately upon loop termination.

SOURCE LISTING: Safe Defensive Branching with Short-Circuit Guard Clauses (C++)

```
#include <iostream>

struct Node {
    int data;
    Node* next;
};

// Defensive processing using short-circuit logical operators
bool processNextNode(Node* current) {
    // Both pointer validity and payload boundary verified safely:
    if (current != nullptr && current->next != nullptr && current->next->data > 0) {
        std::cout << "Valid positive successor: " << current->next->data << "\n";
        return true;
    }
    return false;
}
```

TECHNICAL ANALYSIS

If current is null, the evaluation terminates immediately without evaluating current->next, strictly avoiding segmentation faults.

SAMPLE EXAM QUESTION: Midterm Classic

Trace the output and state of variables in a nested loop with pre-increment vs post-increment operations and state the loop invariant for binary search.

SOLUTION & KEY TAKEAWAY

For binary search in a sorted array `arr[0..n-1]`, the loop invariant is: If the target element `x` exists anywhere within the original array, it **MUST** lie within the sub-index range `[low, high]`. At each iteration, `mid = low + (high - low)/2` divides the search space. By comparing `arr[mid]` to `x`, either `low = mid + 1` or `high = mid - 1` preserves the invariant while monotonically decreasing the search interval size until `low > high`.

CHAPTER 3**Pointers, Pointer Arithmetic & Dynamic Heap Management**

Mastering Memory Dereferencing, Lvalues, Rvalues & Heap Lifecycles

Pointers are the foundational mechanism of systems programming. A pointer variable stores the virtual address of another variable. Understanding pointer arithmetic, multidimensional pointer indexing, function pointers, and the heap allocator (new / delete / malloc / free) is mandatory for systems engineers.

Theoretical Foundation 3.1

Address-of (&) and Dereference (*) operators: & extracts the lvalue base address; * resolves data stored at the targeted address.

Theoretical Foundation 3.2

Pointer Arithmetic: Adding integer k to pointer of type T^* advances the byte address by $k * \text{sizeof}(T)$.

Theoretical Foundation 3.3

Dangling Pointers: Pointers pointing to memory that has already been deallocated via delete or whose stack frame has popped.

Theoretical Foundation 3.4

Memory Leaks: Allocating heap memory without recording its reference or omitting the matching delete statement.

SOURCE LISTING: 2D Dynamic Matrix Allocation with Clean Deallocation Guard (C++)

```
#include <iostream>

int** allocateMatrix(int rows, int cols) {
    int** matrix = new int*[rows];
    for (int i = 0; i < rows; ++i) {
        matrix[i] = new int[cols](); // Zero-initialized
    }
    return matrix;
}

void freeMatrix(int** matrix, int rows) {
    if (!matrix) return;
    for (int i = 0; i < rows; ++i) {
        delete[] matrix[i]; // Free each row vector
    }
    delete[] matrix; // Free top pointer array
}

int main() {
    const int R = 3, C = 4;
    int** mat = allocateMatrix(R, C);
    mat[1][2] = 99;
    std::cout << "Value at mat[1][2]: " << mat[1][2] << "\n";
    freeMatrix(mat, R);
    return 0;
}
```

TECHNICAL ANALYSIS

Demonstrates complete 2D array allocation on the heap and prevents memory leaks by deallocating in reverse allocation order.

SAMPLE EXAM QUESTION: Final Exam Mastery

Differentiate between `int* const p` vs `const int* p` vs `const int* const p`. What operations are allowed or rejected on each?

SOLUTION & KEY TAKEAWAY

- 1) `const int* p` (or `int const* p`): Pointer to constant integer. The integer data being pointed to cannot be altered through `p` (`*p = 10` is illegal), but `p` itself can be redirected to point to another address (`p = &y` is legal).
- 2) `int* const p`: Constant pointer to mutable integer. The address stored inside `p` cannot be changed (`p = &y` is illegal), but the integer value at that address may be modified (`*p = 10` is legal).
- 3) `const int* const p`: Constant pointer to constant integer. Neither the targeted value (`*p = 10`) nor the target address (`p = &y`) may ever be modified.

LECTURE NOTES

CS-101 Slide Deck: Core Modules 1 & 2

Classroom Lecture Companion

SLIDE 1: MODULE 1: MACHINE ARCHITECTURE & MEMORY LAYOUT

- Von Neumann architecture: CPU, RAM, Cache hierarchies & Bus layout
- Compilation pipeline: Preprocessor -> Compiler -> Assembler -> Linker
- Variables as memory addresses: Hexadecimal offsets and bitwidth representations
- Primitive types: int (4 bytes), char (1 byte), float (4 bytes), double (8 bytes)

SLIDE 2: MODULE 2: CONTROL FLOW & ALGORITHMIC BRANCHING

- Short-circuit evaluation in logical operators (&&, ||)
- Switch-case jump tables vs cascaded if-else performance trade-offs
- Loop invariants, boundary conditions, and accumulator patterns
- Break vs continue within nested multi-dimensional iteration

SLIDE 3: MODULE 3: FUNCTIONS, STACK FRAMES & RECURSION

- Call stack layout: Return address, saved frame pointer, local variables
- Pass-by-value makes a copy; pass-by-reference (&) passes direct memory alias
- Base cases prevent stack overflow; recurrence relations capture repetitive state
- Function overloading: Signature resolution determined by parameter types

SLIDE 4: MODULE 4: POINTERS & DYNAMIC HEAP MEMORY

- Stack memory (automatic lifecycle) vs Heap memory (manual lifecycle with new/delete)
- Pointer arithmetic: `ptr + 1` advances by `sizeof(*ptr)` bytes in memory
- Dangling pointers, wild pointers, and double-free exceptions
- Dynamic 2D arrays: Array of pointers vs contiguous single-block allocation

KEY EXAM HIGHLIGHTS FROM LECTURES

1. Always draw memory box diagrams for pointer dereferencing (*p) and address-of (&x) tracing questions.
2. Watch out for infinite loop edge conditions and off-by-one errors in nested matrix loops.
3. Check dynamic memory allocation for leaks: each new must have a corresponding delete.

LAB & PRACTICUM CHECKLIST

Project Option 1: Student Record Management System with File I/O

Project Option 2: Console-based Battleship Game with Procedural AI

Project Option 3: CLI Banking Ledger with Transaction Logging

CHAPTER 6

CS-101 Midterm Examination Solved Bank

Step-by-step Model Solutions

MIDTERM PROBLEM [Midterm Standard]

Explain the exact sequence of events when a recursive function reaches stack overflow.

OFFICIAL MODEL SOLUTION

Each invocation pushes a stack frame containing: return instruction address, incoming arguments, saved CPU registers (frame pointer RBP), and local variables. If the base case is missing or unreachable, the stack pointer (RSP) decrements into the unmapped guard page below the stack boundary, triggering a hardware MMU fault reported as SIGSEGV (Segmentation Fault).

CHAPTER 6 (CONT.)**CS-101 Final Examination Mastery**

Comprehensive Rubrics

EXAMINATION PROTOCOL & SCORING RUBRICS

1. Allocate time proportionally to marks.
2. Write formal preconditions and loop invariants.
3. Show scratch work for memory diagrams.
4. Clearly state time and auxiliary space complexity.

CHAPTER 7

CS-101 Semester Capstone Lab Specification

Hands-on Production Implementation

GRADING CRITERIA & DEMO EXPECTATIONS

- Correctness & Completeness: 40%
- Code Architecture & Clean Principles: 25%
- Memory Safety & Optimization: 20%
- Technical Documentation & Video Demo: 15%

CHAPTER 8**CS-101 Technical Interview Preparation**

FAANG & High-Growth Tech Interview Questions

INTERVIEW SCENARIO: Systems Performance & Memory Security

Why does C++ have undefined behavior, and what happens when you write beyond array bounds?

OPTIMAL ARCHITECTURAL SOLUTION

C++ omits mandatory runtime bounds checking to maximize raw execution velocity. Writing beyond an array boundary corrupts neighboring stack or heap variables, overwrites saved return addresses (stack smashing), or attempts to write into read-only memory pages, leading to erratic bugs or exploitable security vulnerabilities (CVE buffer overflows).

INTERVIEW CHECKLIST

1. Clarify constraints and expected volume.
2. Explain algorithmic intuition before typing.
3. Identify space-time trade-offs.
4. Walk through an edge case manually.

CHAPTER 9

CS-101 Lexicon & Authoritative References

Glossary of Formal Terms and Literature

Key Term	Rigorous Academic Definition
Lvalue	An expression that designates a persistent memory location and can appear on the left side of an assignment.
Rvalue	A temporary value that does not persist beyond the expression that computed it.
Stack Frame	A contiguous block of memory pushed onto the runtime stack during a function call.
Heap Fragmentation	Condition where free memory is broken into non-contiguous slices, preventing large contiguous allocations.

- RECOMMENDED TEXTBOOKS
1. C++ How to Program - Paul Deitel & Harvey Deitel

2. Starting Out with C++: From Control Structures through Objects - Tony Gaddis

3. The C Programming Language - Brian Kernighan & Dennis Ritchie